HCMC University of Technology, VNU



Software Architecture

Chapter 2. SOLID Principles

Lecturer: Phat T. Tran-Truong, Meng.

Email: phatttt@hcmut.edu.vn

CHAPTER 2. SOLID Principles



2.1. **S**ingle Responsibility Principle

2.2. **O**pen-Closed Principle

2.3. **L**iskov Substitution Principle

2.4. **I**nterface Segregation Principle

2.5. **D**ependency Inversion Principle

What is SOLID principles?



- Good software systems begin with clean code. On the one hand, if the bricks aren't well made, the architecture of the building doesn't matter much. On the other hand, you can make a substantial mess with well-made bricks. This is where the SOLID principles come in.
- **SOLID** principles tell us how to arrange our functions and data structures into classes, and how those classes should be interconnected.
- The goal of the principles is the creation of mid-level software structures that: *Tolerate change; Are easy to understand* and *Are the basis of components that can be used in many software systems*.

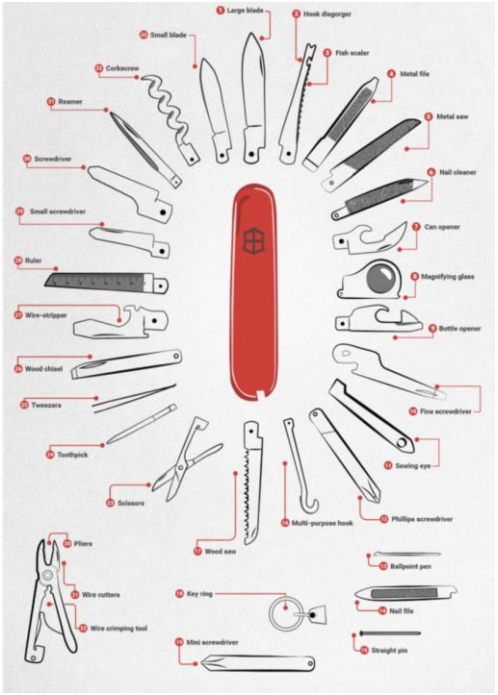
SRP: The Single Responsibility Principle



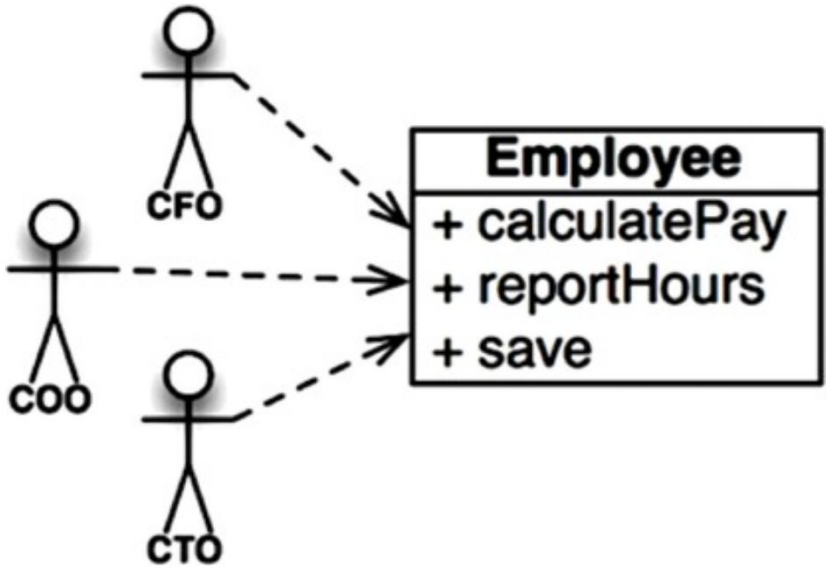
An active corollary to **Conway's law**: The best structure for a software system is heavily influenced by the social structure of the organization that uses it so that each software **module has one, and only one, reason to change**.



A module should be responsible to one, and only one, actor.



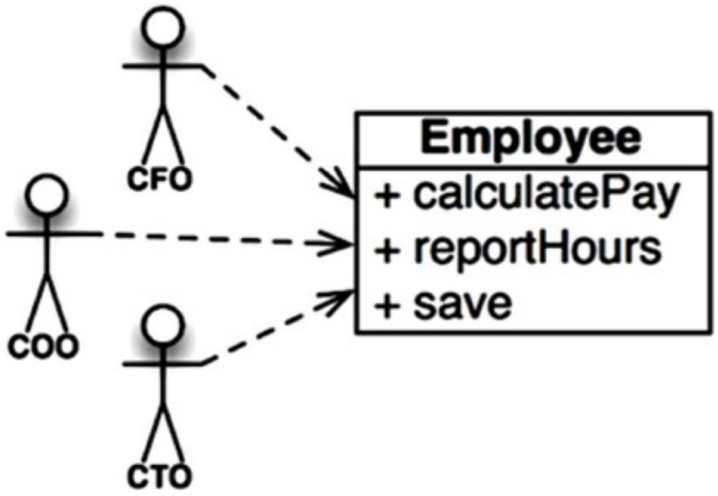
SRP: Case-study

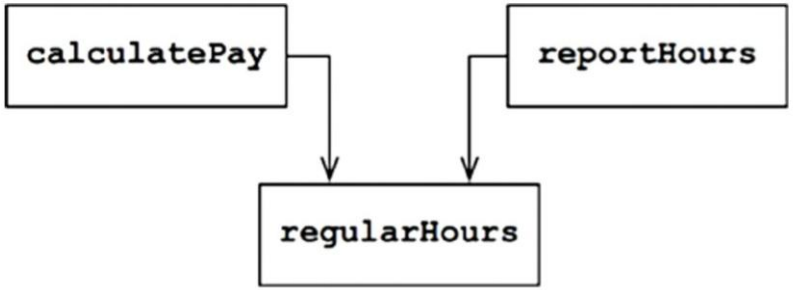


The Employee class has 3 methods that reports to three different actors: CFO, COO & CTO. If one of these actors modifies their function, Employee is modified and the other actors are also affected.

=> Anti-pattern: *A module is responsible to more than actors.*

SRP: Case-study





Eg:

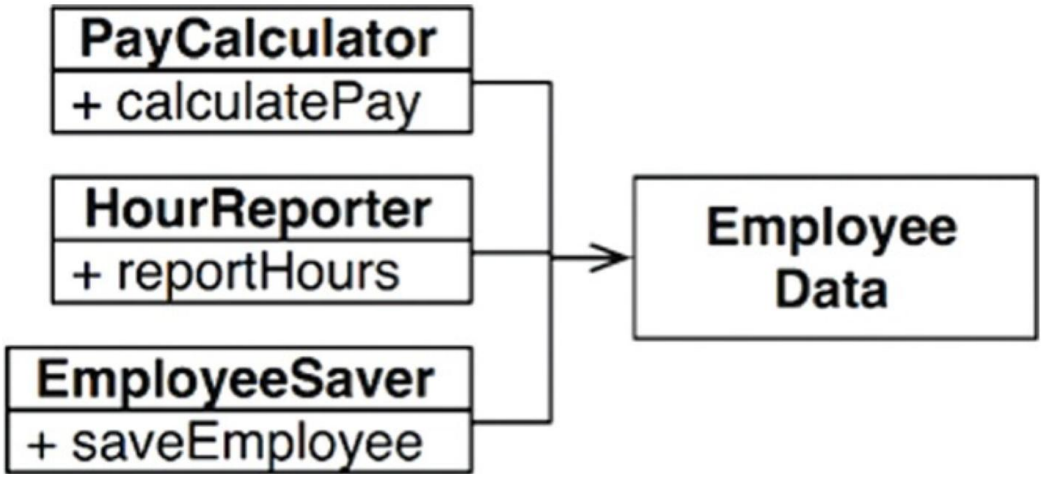
- Suppose that the *calculatePay()* function and the *reportHours()* function share a common algorithm for calculating non-overtime hours – *reportHours().*
- Now, CFO's team decides that the way non-overtime hours are calculated needs to be tweaked (but COO's team don't want).
- Developers makes the required changes for CFO and they both don't know about COO also use this *reportHours()* for other purposes. Besides, COO's team even don't acknowledge their functions have been modified.

=> Ensure SRP: *A module should be responsible to one, and only one, actor.*

SRP: Solution (1)



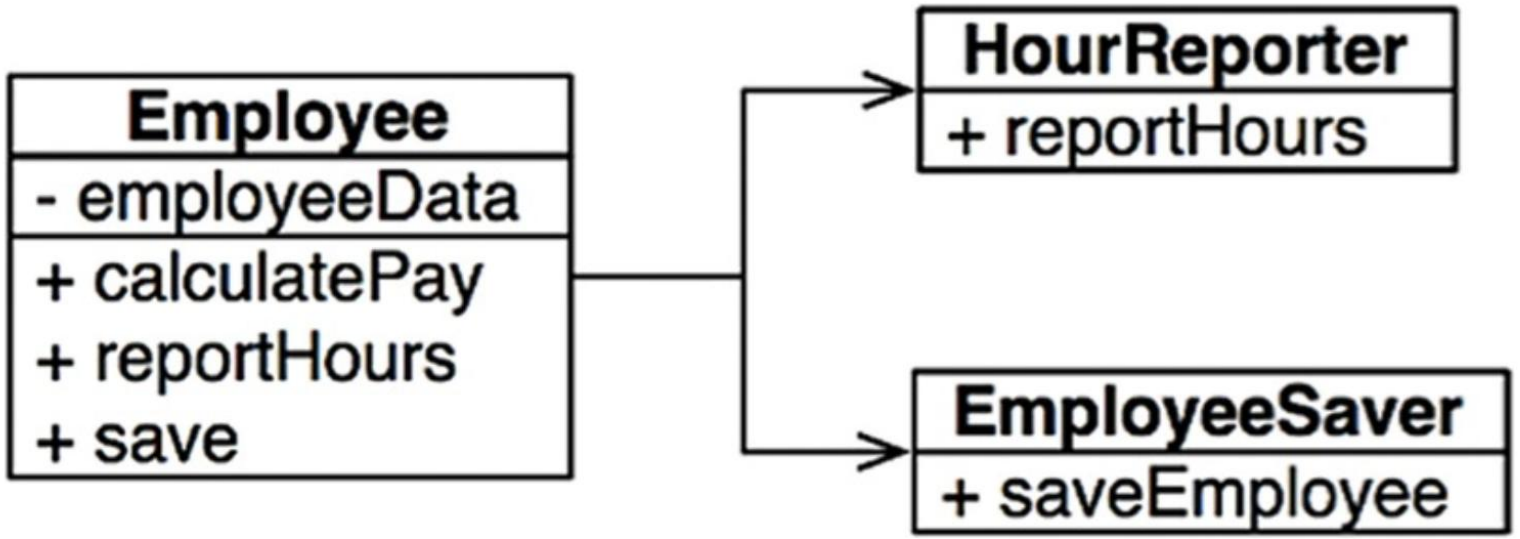
- We can refactor 3 methods into 3 separated classes that do not know about each other. These classes access to EmployeeData independently for retrieving employee data to perform their behaviors.



SRP: Solution (1)



- Some developers prefer to keep the most important business rules closer to the data. The most important method (eg: save) is kept in the orginal *Employee* class and used as *Facade* for the lesser functions.



SRP: Solution (2)



- The downside of this solution is that the developers now have three classes that they have to instantiate and track. A common solution to this dilemma is to use the ***Facade* pattern**

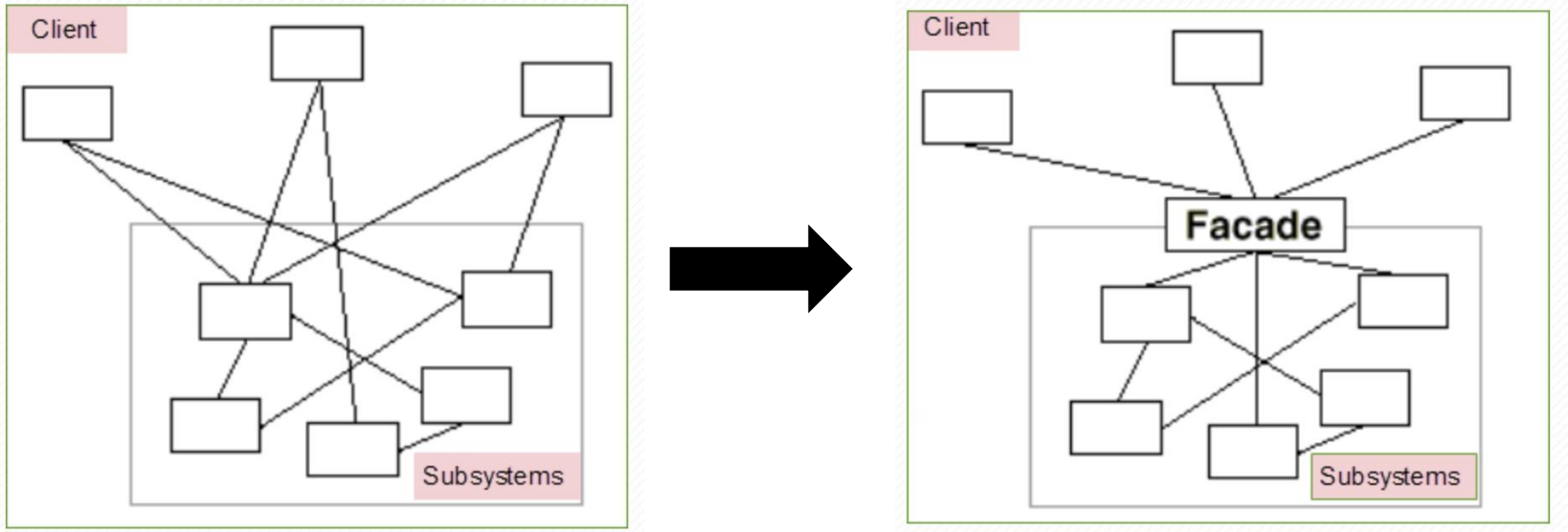


Figure 7.4 The Facade pattern

SRP: Solution (2) – Facade pattern

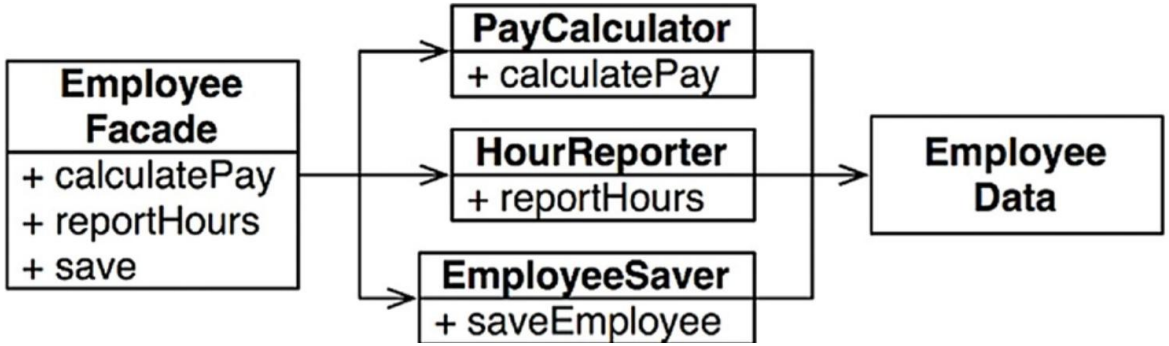


- **Facade** is a structural design pattern that provides a simplified (but limited) interface to a complex system of classes, library or framework.



- If the subsystems are complex, we can create may facades (but avoid too much and non-sense).

SRP: Example




```
public class EmployeeData {
    public Employee getEmployeeData();
}
```

```
public class PayCalculator {
    private EmployeeData empData;

    PayCalculator(EmployeeData empData){
        this.empData = empData;
    }

    public void calculatePay(){
        empData.getEmployeeData();
        //calculatePay();
    }
}
```

```
public class HourReporter {
    private EmployeeData empData;

    HourReporter(EmployeeData empData){
        this.empData = empData;
    }

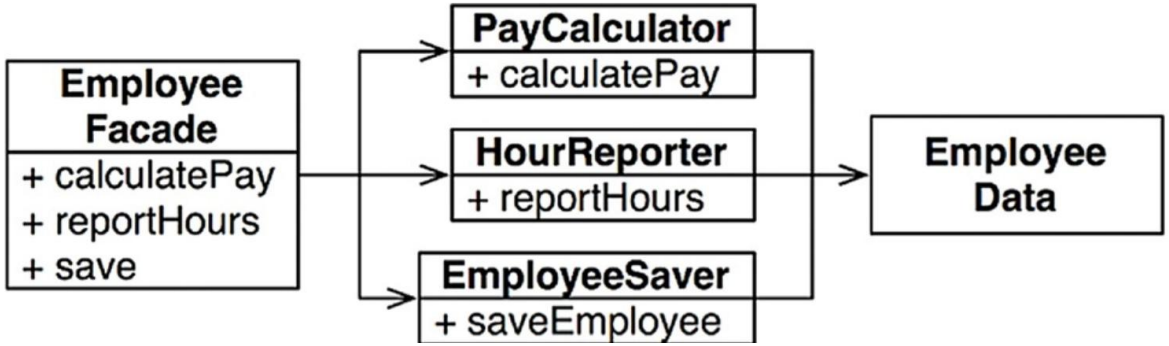
    public void reportHours(){
        empData.getEmployeeData();
        //reportHours();
    }
}
```

```
public class EmployeeSaver {
    private EmployeeData empData;

    EmployeeSaver(EmployeeData empData){
        this.empData = empData;
    }

    public void saveEmployee(){
        empData.getEmployeeData();
        //saveEmployee();
    }
}
```

SRP: Example




```java
public class EmployeeFacade {
    private PayCalculator payCalculator;
    private HourReporter hourReporter;
    private EmployeeSaver employeeSaver;

    //Constructors
    EmployeeFacade(){
        payCalculator = new PayCalculator();
        hourReporter = new HourReporter();
        employeeSaver = new EmployeeSaver();
    }

    public void calculatePay(){
        payCalculator.calculatePay();
    }

    public void reportHours(){
        hourReporter.reportHours();
    }

     public void save(){
        employeeSaver.save();
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        EmployeeFacade employeeFacade = new EmployeeFacade();

        employeeFacade.calculatePay();
        employeeFacade.reportHours();
        employeeFacade.save();
    }
}
```

SRP: Conclusion



- The Single Responsibility Principle is about functions and classes—but it reappears in a different form at two more levels. At the level of components, it becomes the **Common Closure Principle**. At the architectural level, it becomes the **Axis of Change** responsible for the creation of Architectural Boundaries.

SRP: Exercise 1



- Each student gives 01 case of violation of SRP principle and proposes a solution.

SRP: Exercise 2



Design & implement the following context to satisfy SRP:

- Let's consider a hotel. This hotel has a hotel keeper. There are a lot of restaurants inside the hotel e.g. Vegetarian restaurants, Non-Vegetarian restaurants, and Omnivore restaurants.
  - You, as a client want access to different menus of different restaurants. You do not know what are the different menus they have. You just have access to a hotel keeper who knows his hotel well.
  - Whichever menu you want, you tell the hotel keeper and he takes it out of the respective restaurants and hands it over to you.

CHAPTER 2. SOLID Principles



2.1. **S**ingle Responsibility Principle

2.2. **O**pen-Closed Principle

2.3. **L**iskov Substitution Principle

2.4. **I**nterface Segregation Principle

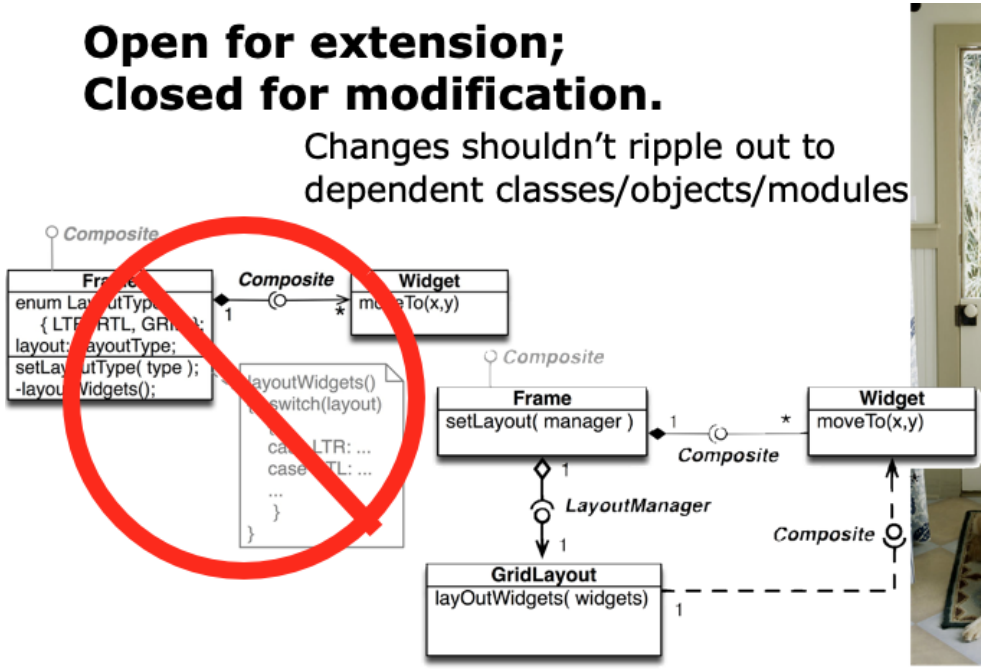
2.5. **D**ependency Inversion Principle

OCP: The Open-Closed Principle



The Open Closed Principle (OCP) was coined in 1988 by Bertrand Meyer. It says:

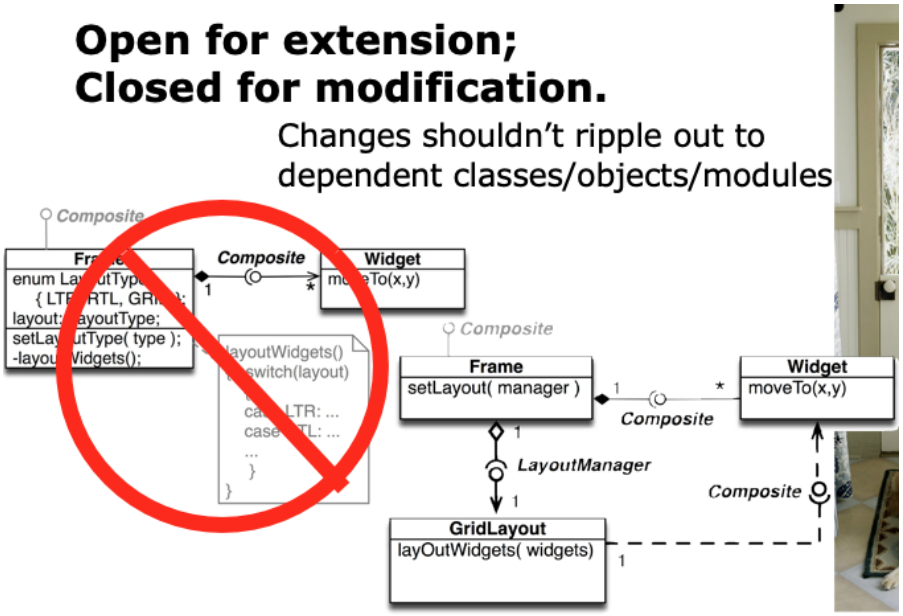
- *A software artifact should be open for extension but closed for modification.*



OCP: The Open-Closed Principle



Most students of software design recognize the OCP as a principle that guides them in the design of classes and modules. But the principle takes on even greater significance when we consider the level of architectural components.



OCP: Case-study



- We have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.
- The stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.

OCP: Case-study



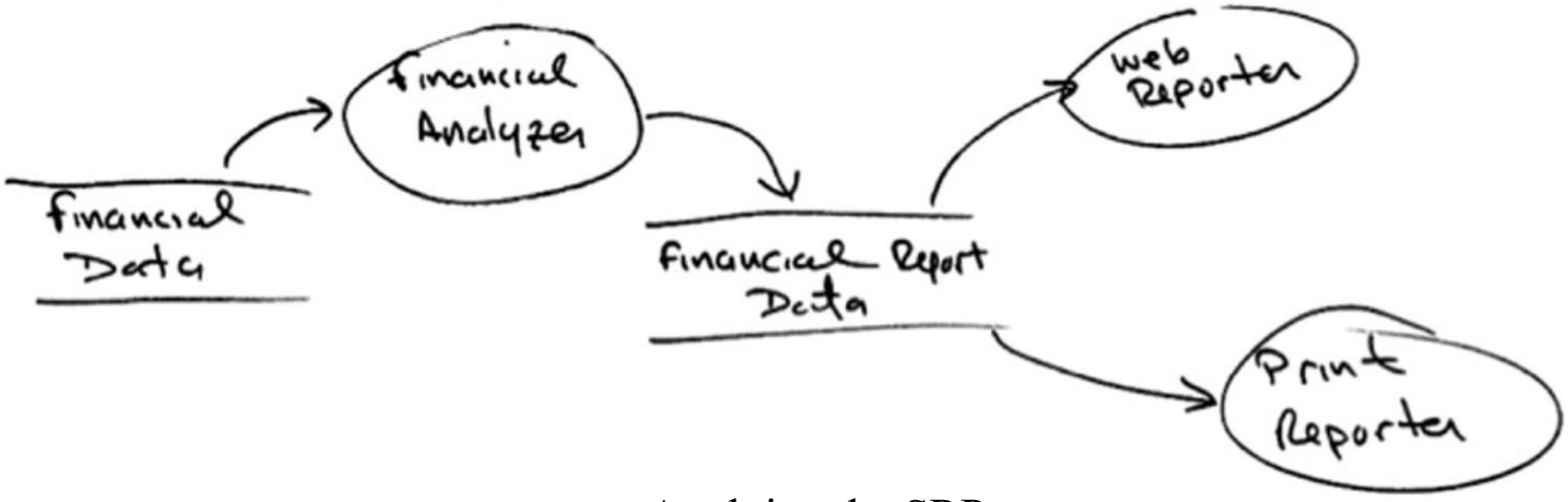
- We have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.
- The stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.
- Clearly, some new code must be written. But how much old code will have to change?

OCP: Case-study



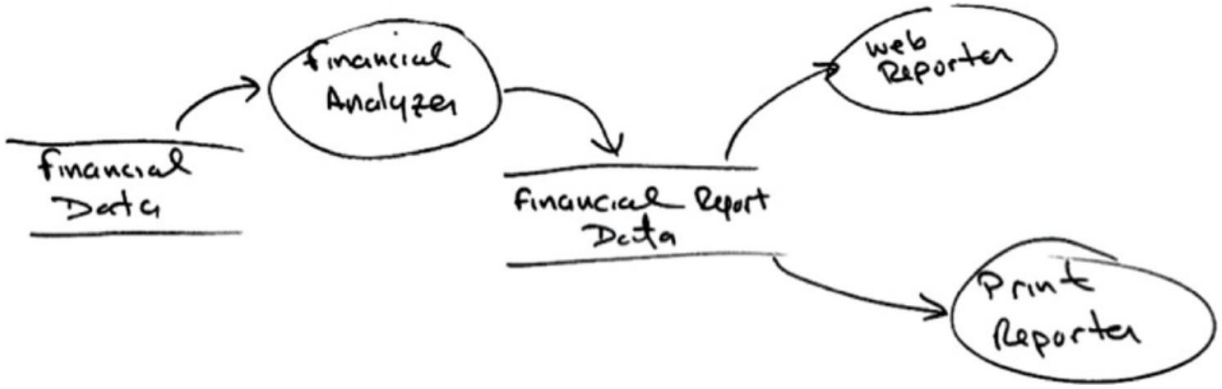
- Clearly, some new code must be written. But how much old code will have to change?
- A good software architecture would reduce the amount of changed code to the barest minimum. Ideally, **zero**.
- How? By properly separating the things that change for different reasons (the Single Responsibility Principle), and then organizing the dependencies between those things properly (the Dependency Inversion Principle).

OCP: Case-study



Applying the SRP

OCP: Case-study



Applying the SRP

- The essential insight here is that generating the report involves two separate responsibilities: the calculation of the reported data, and the presentation of that data into a web- and printer-friendly form.
- Having made this separation, we need to organize the source code dependencies to ensure that changes to one of those responsibilities do not cause changes in the other. Also, the new organization should ensure that the behavior can be extended without undo modification.

OCP: Solution (1)



If component A should be **protected from changes** in component B, then component B should **depend on** component A.

Eg: From above analyses, we can design which components are “equal”, which are higher ones, which are protected from changes from which ones, and which ones depend on others.

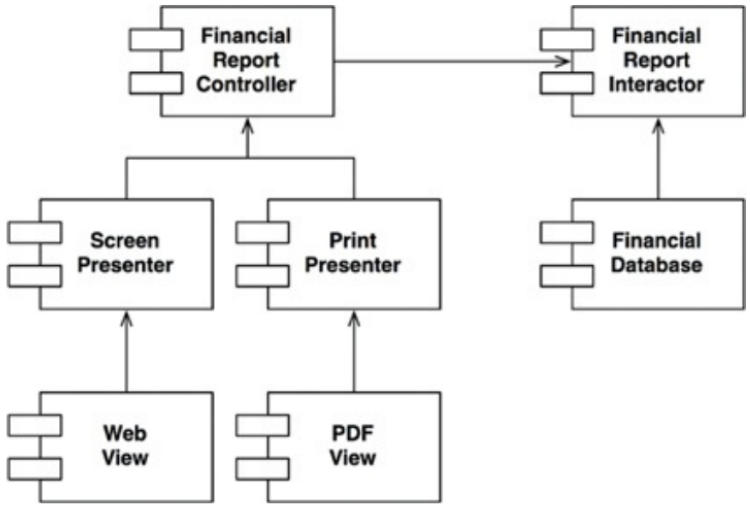


Figure 8.3 The component relationships are unidirectional

OCP: Solution (1)



- The *Interactor* is in the position that best conforms to the OCP. Changes to the *Database*, or the *Controller*, or the *Presenters*, or the *Views*, will have no impact on the *Interactor*.

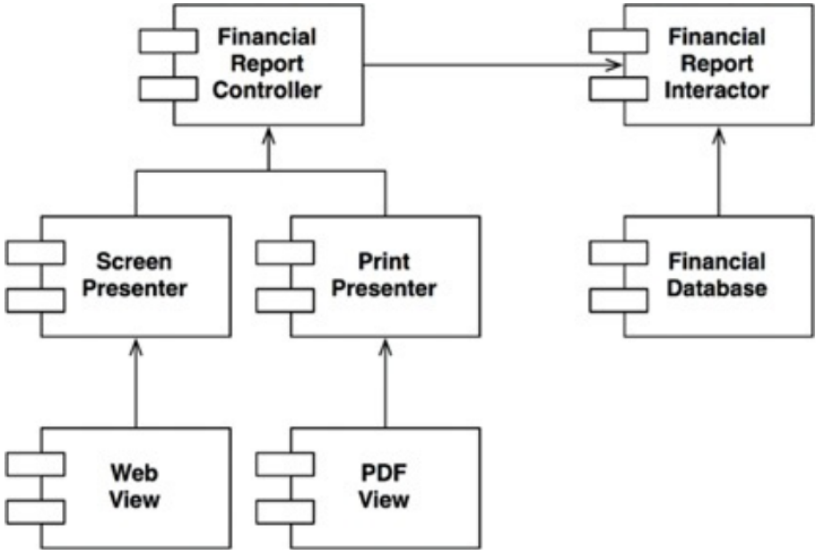


Figure 8.3 The component relationships are unidirectional

- This is how the OCP works at the architectural level. Architects separate functionality based on how, why, and when it changes.

OCP: Solution (1)

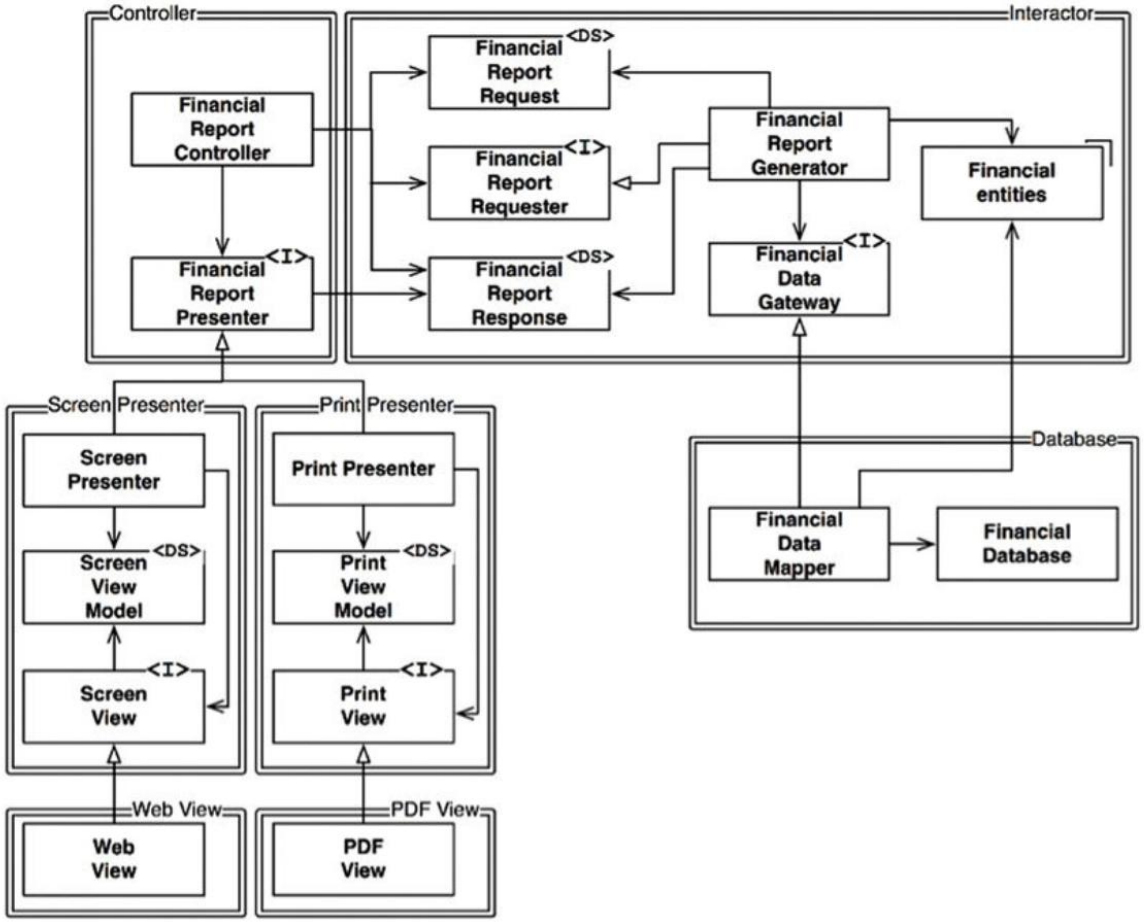


Figure - Partitioning the processes into classes and separating the classes into components

OCP: Solution (1)



- The OCP is one of the driving forces behind the architecture of systems. The goal is to make the system **easy to extend without incurring a high impact of change**. This goal is accomplished by partitioning the system into **components**, and arranging those components into a dependency hierarchy that **protects** higher-level components from changes in lower-level components.

OCP: Solution (2) - Strategy Pattern

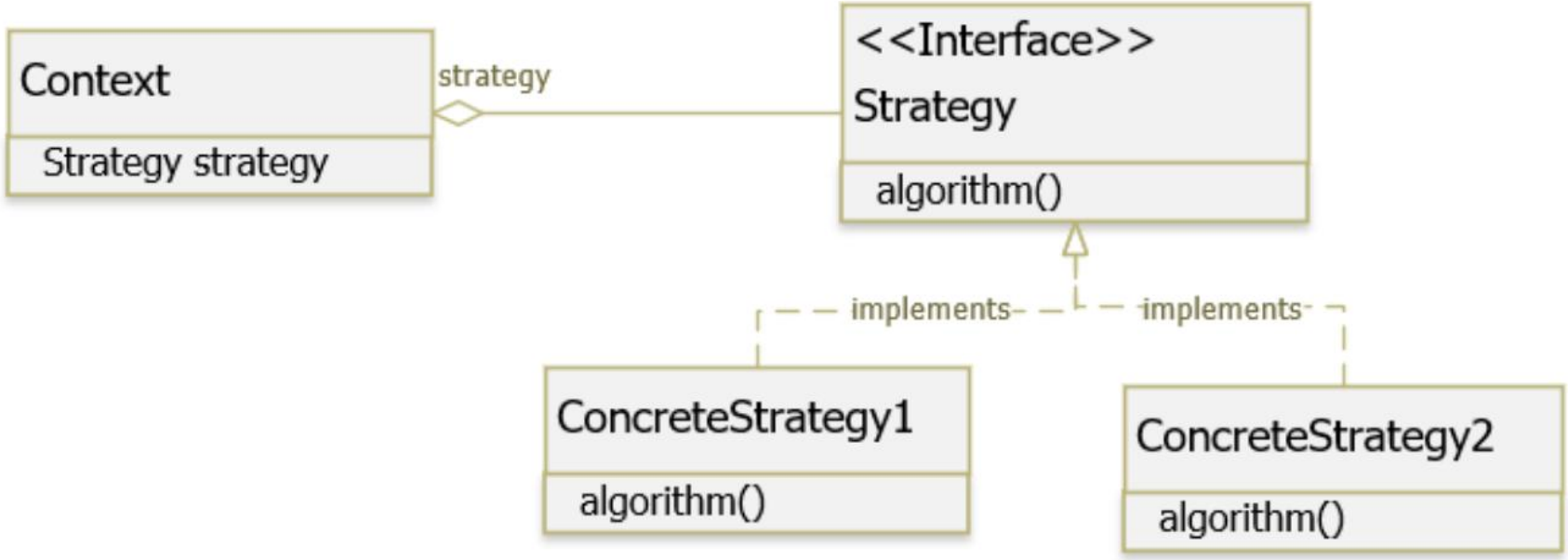


- System components (classes, modules, functions) should be **open for extension** but **closed for modification**. That is, you should be able to extend the functionality of a component without changing its existing source code. This can be achieved through inheritance, using interfaces, or design patterns like the **Strategy Pattern**.
- Eg: instead of modifying a class directly to add new functionality, you can extend it through inheritance.

OCP: Solution (2) - Strategy Pattern



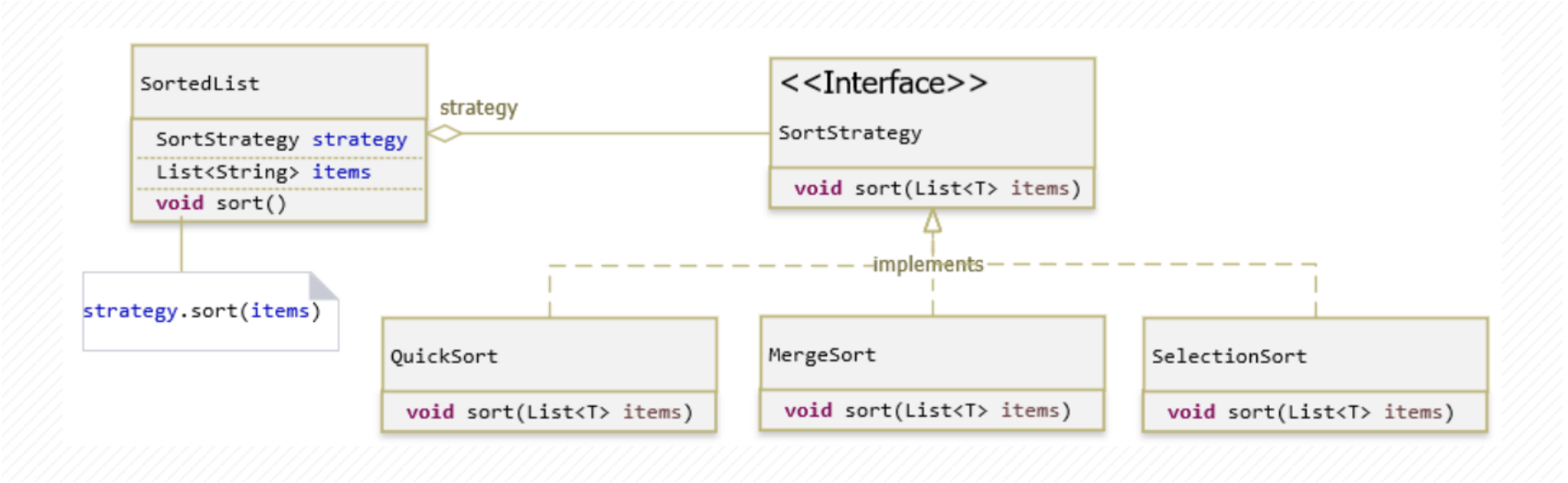
- Behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects.



OCP: Solution (2) - Strategy Pattern



- Example:



OCP: Exercise 1

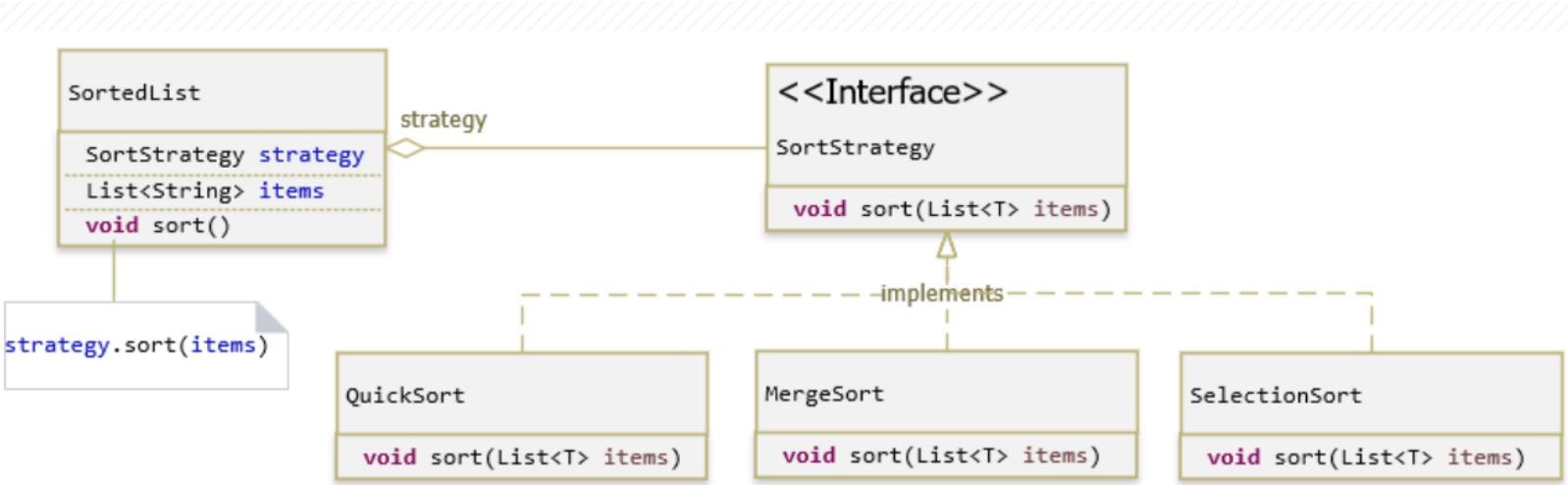


- Each student gives 01 case of OCP principle.

OCP: Exercise 2



- Students implement the following Strategy pattern for sorting algorithms by an OOP programming language, such as Java.



CHAPTER 2. SOLID Principles



2.1. **S**ingle Responsibility Principle

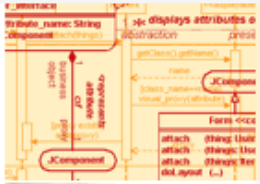
2.2. **O**pen-Closed Principle

2.3. **L**iskov Substitution Principle

2.4. **I**nterface Segregation Principle

2.5. **D**ependency Inversion Principle

LSP: The Liskov Substitution Principle



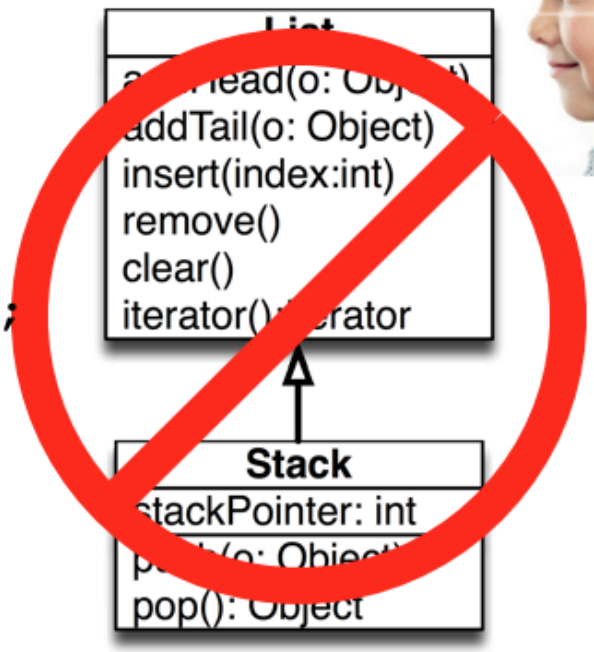
Liskov Substitution Principle



Subtypes must be substitutable for their base types.

```
f( List l )
{
  l.clear();

  l.addHead("x");
}
Stack aStack...;
f(aStack);
```





Context Matters!

LSP: The Liskov Substitution Principle



- Objects of a child class must be able to replace the parent class without changing the correctness of the program. In other words, the child class must maintain the behavior of the parent class and not change the logic when replacing it.
- Eg: if you have a Shape class, child classes such as Rectangle or Circle must be able to be used in places where the Shape class is used without causing errors in the system.



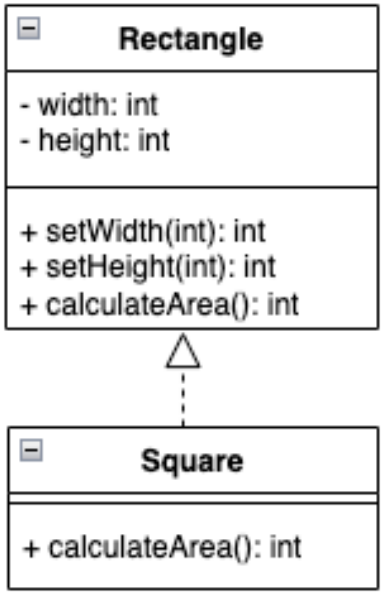
LSP: The Liskov Substitution Principle



- In the early years of the object-oriented revolution, we thought of the LSP as a way to guide the use of inheritance, as shown in the previous sections. However, over the years the LSP has morphed into a broader principle of software design that pertains to interfaces and implementations.



LSP: Violation




```java
package com.gpcoder.solid;

/**
 * LSP - Liskov substitution principle example
 *
 * @author gpcoder
 */
class Rectangle {
    private int width;
    private int height;

    public int calculateArea() {
        return this.width * this.height;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }
}

class Square extends Rectangle {

    @Override
    public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
    }

    @Override
    public void setHeight(int height) {
        super.setWidth(height);
        super.setHeight(height);
    }
}
```

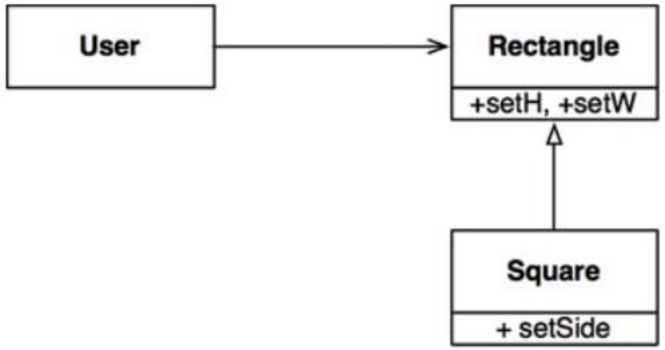
```java
public class LSPExample1 {
    public void example1() {
        Rectangle rect = new Rectangle();
        rect.setWidth(5);
        rect.setHeight(10);
        System.out.println(rect.calculateArea()); // 50

        Square square = new Square();
        square.setWidth(5);
        square.setHeight(10);
        System.out.println(square.calculateArea()); // 100
    }
}
```

LSP: Violation



- Example LSP Violation



- In this example, Square is not a proper subtype of Rectangle because the height and width of the Rectangle are independently mutable; in contrast, the height and width of the Square must change together. Since the User believes it is communicating with a Rectangle, it could easily get confused.

LSP: Conclusion



- The LSP can, and should, be extended to the level of architecture. A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms.

CHAPTER 2. SOLID Principles



2.1. **S**ingle Responsibility Principle

2.2. **O**pen-Closed Principle

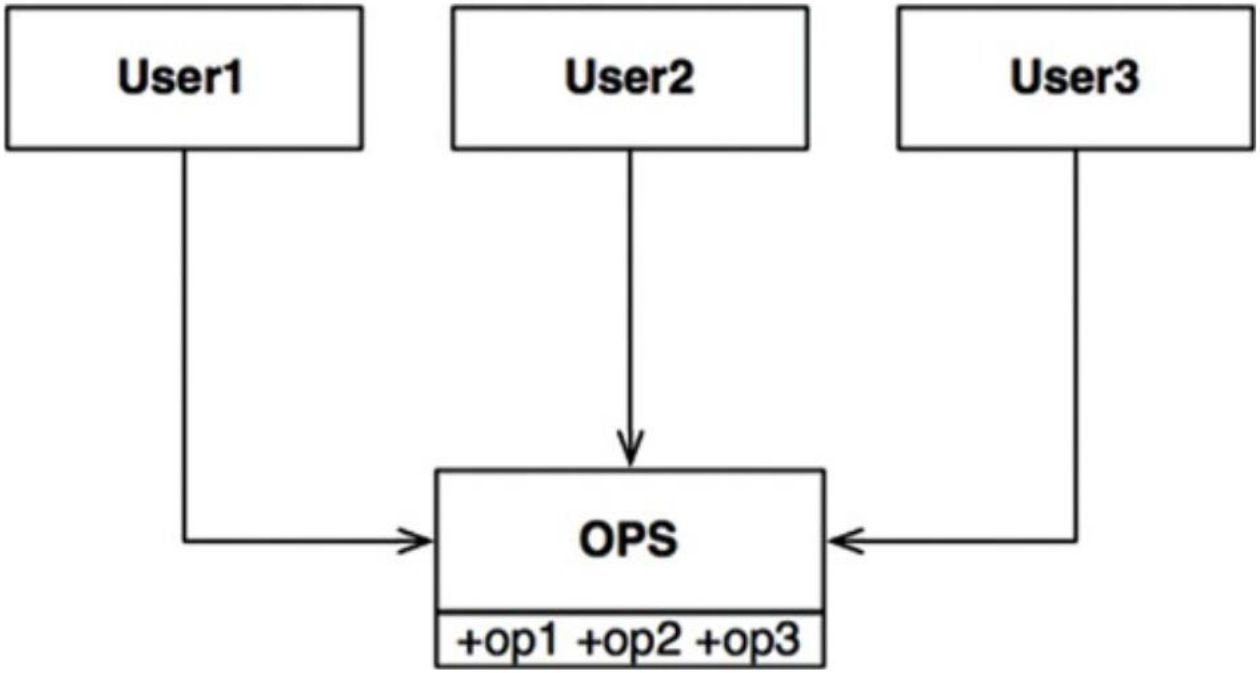
2.3. **L**iskov Substitution Principle

2.4. **I**nterface Segregation Principle

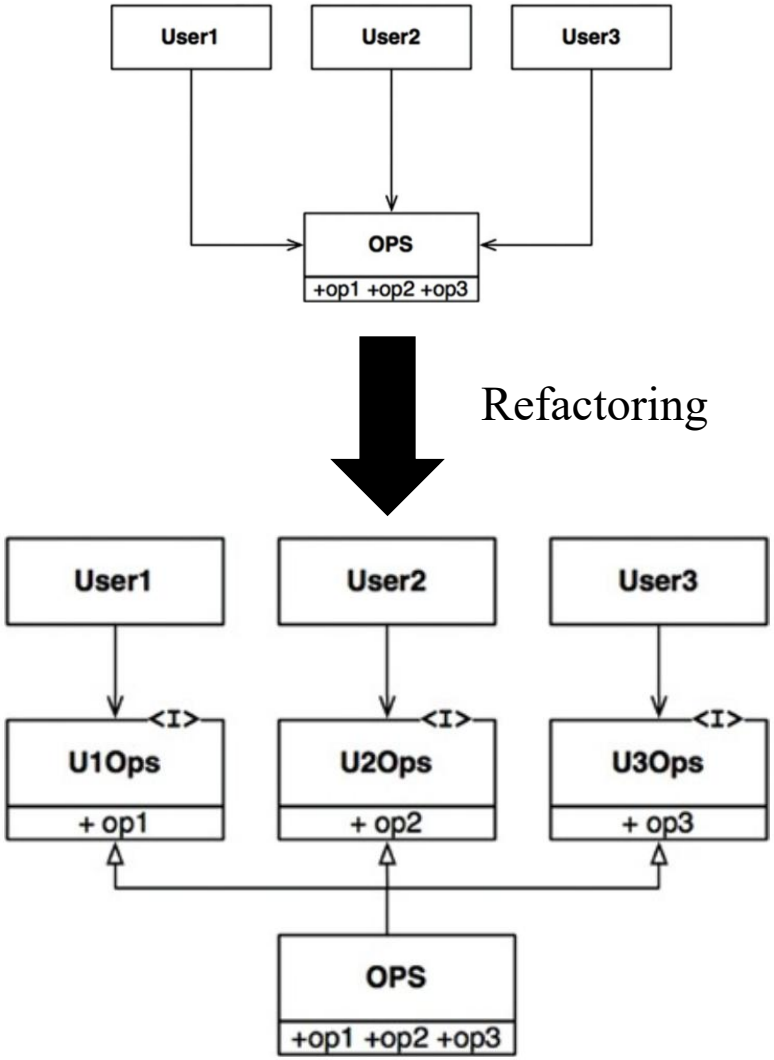
2.5. **D**ependency Inversion Principle

ISP: The Interface Segregation Principle





ISP: The Interface Segregation Principle



ISP: Example



```
public interface Animal {
    void Eat();
    void Drink();
    void Sleep();
    //...
    void Swim();
     void Fly();
}
```

```
public class Cat implements Animal{

    public void Eat() {

    public void Drink() {

    public void Sleep() {

    @Override
    public void Swim() {
        // TODO Auto-generated method stub

    }

    @Override
    public void Fly() {
        // TODO Auto-generated method stub

    }

}
```

Class Cat doesn’t have Swim() and Fly() method!

ISP: Example



- Segregate into 2 minimum interfaces Animal and Bird

```
public interface Animal {
    void Eat();
    void Drink();
    void Sleep();

}
```

```
2  public interface Bird {
3      void Fly();
4  }
5
```

- Each subclass implements necessary interfaces itself.

```
2  public class Cat implements Animal{
3
4      @Override
5      public void Eat() {
6          // TODO Auto-generated method stub
7
8      }
9
0      @Override
1      public void Drink() {
2          // TODO Auto-generated method stub
3
4      }
5
6      @Override
7      public void Sleep() {
8          // TODO Auto-generated method stub
9
0      }
1
2  }
```

```
2  public class Dove implements Animal, Bird
3  {
4
6  public void Eat() {
0
2  public void Drink() {
6
8  public void Sleep() {
2
3  @Override
4  public void Fly() {
5      // TODO Auto-generated method stub
6
7  }
8
9  }
```

ISP: The Interface Segregation Principle



- **ISP and language**
  - Statically typed languages like Java force programmers to create declarations that users must *import*, or *use*, or otherwise *include*. It is these included declarations in source code that create the source code dependencies that force recompilation and redeployment.
  - In dynamically typed languages like Ruby and Python, such declarations don't exist in source code. Instead, they are inferred at runtime. Thus there are no source code dependencies to force recompilation and redeployment.

ISP: The Interface Segregation Principle



- **ISP and architecture**



Figure 10.3 A problematic architecture

- Suppose that D contains features that F does not use and, therefore, that S does not care about. Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S. Even worse, a failure of one of the features within D may cause failures in F and S.

=> In general, it is harmful to depend on modules that contain more than you need.

ISP: Conclusion



- The lesson here is that depending on something that carries baggage that you don’t need can cause you troubles that you didn’t expect.
- Interfaces should be small and specific, so that classes only need to implement the methods they really need. Avoid creating large interfaces with lots of unrelated methods that the class doesn't use.

ISP: Exercise



- Each student gives 01 case of ISP principle violation and refactor it.

CHAPTER 2. SOLID Principles



2.1. **S**ingle Responsibility Principle

2.2. **O**pen-Closed Principle

2.3. **L**iskov Substitution Principle

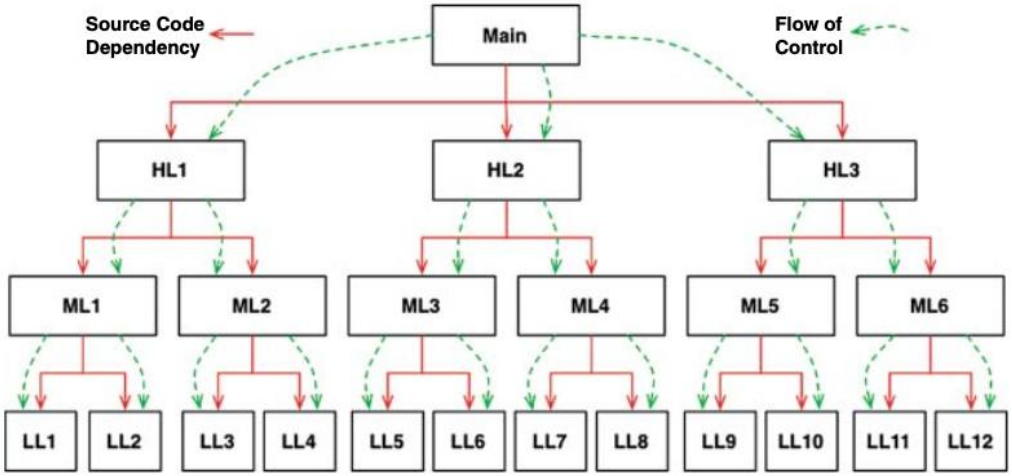
2.4. **I**nterface Segregation Principle

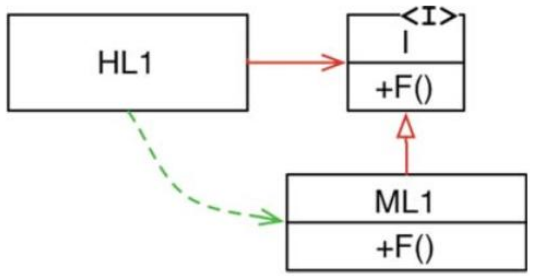
2.5. **D**ependency Inversion Principle

DIP: The Dependency Inversion Principle



The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code **dependencies** refer only to **abstractions**, not to **concretions**.



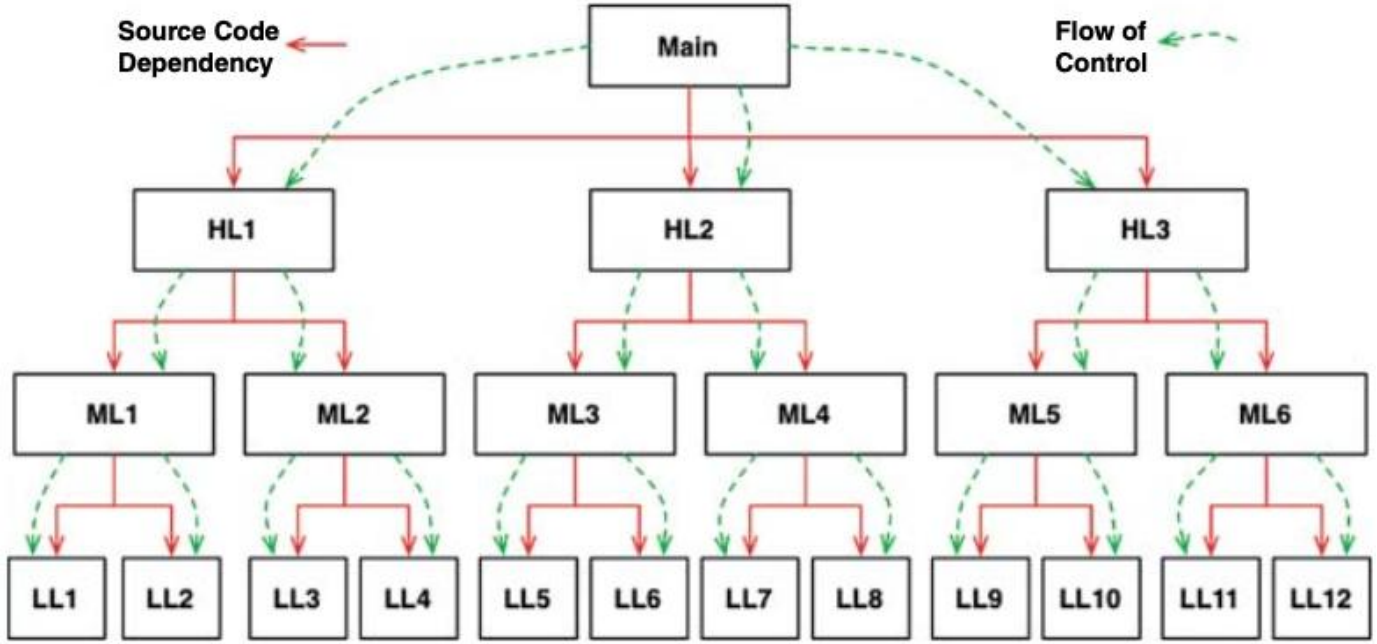


DIP: The Dependency Inversion Principle

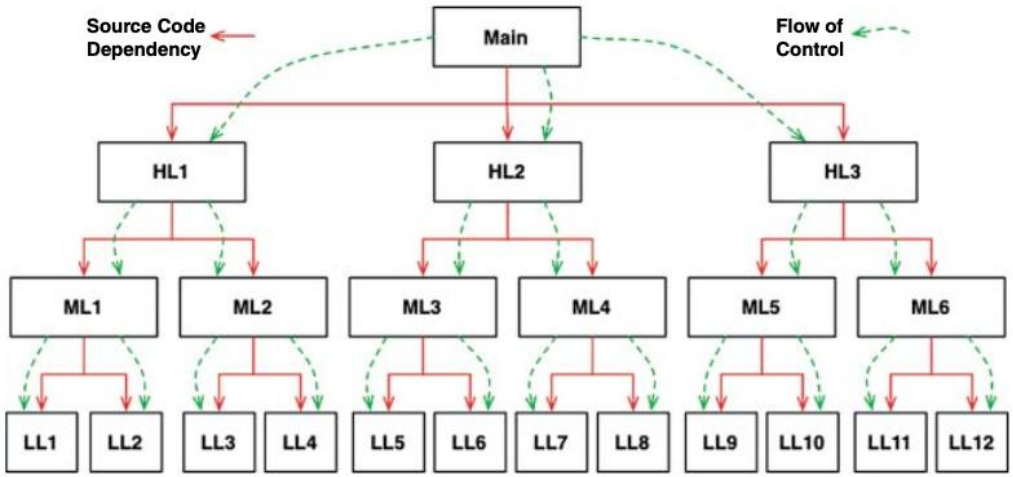


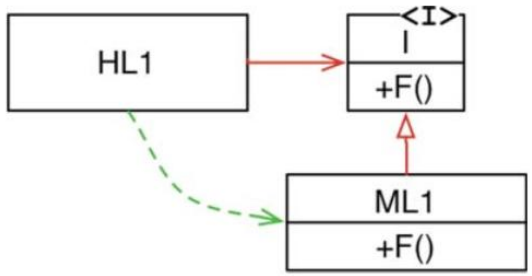
What is Dependency Inversion?

- In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions. In that calling tree, however, source code dependencies inexorably followed the flow of control.



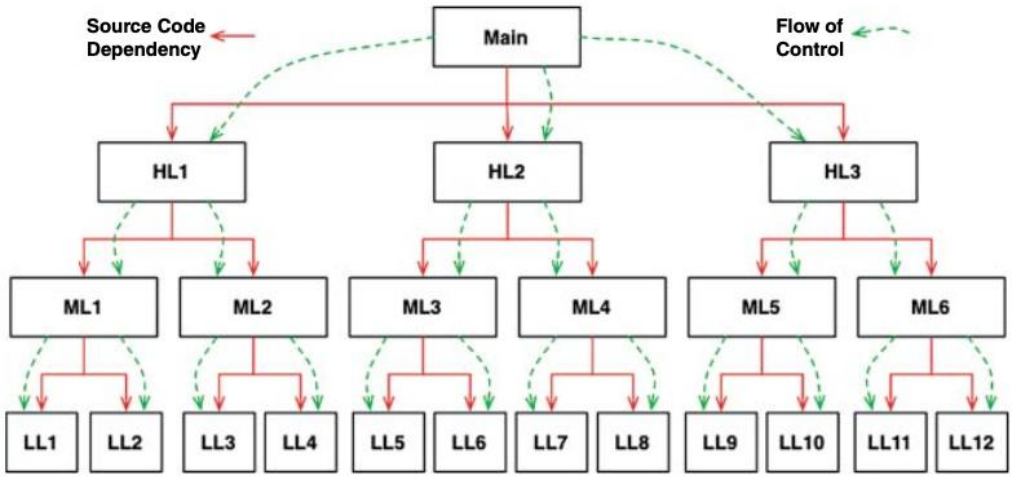
DIP: The Dependency Inversion Principle

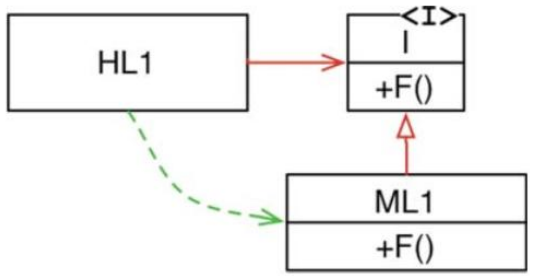




- In right Figure, module HL1 calls the F() function in module ML1. The fact that it calls this function through an interface is a source code contrivance. At runtime, the interface doesn't exist. HL1 simply calls F() within ML.

DIP: The Dependency Inversion Principle





- In fact, the flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.
- The source code dependency (the inheritance relationship) between ML1 and the interface I points in the opposite direction compared to the flow of control.

=> Dependency inversion.

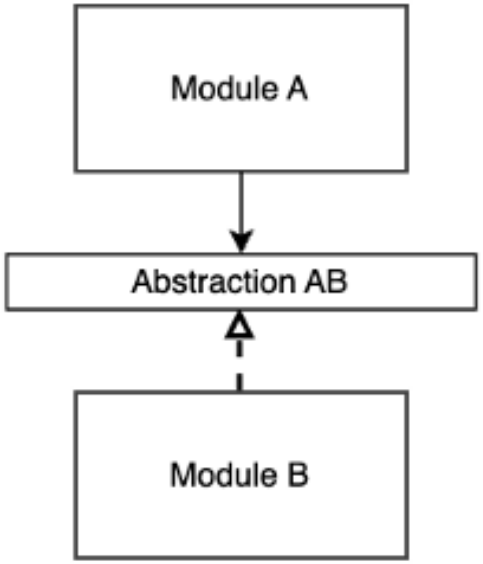
- *In OOP, any source code dependency, no matter where it is, can be inverted.*

DIP: The Dependency Inversion Principle



- Declarations:

> - *High-level modules should not depend on low-level modules. Both should depend on abstractions.*



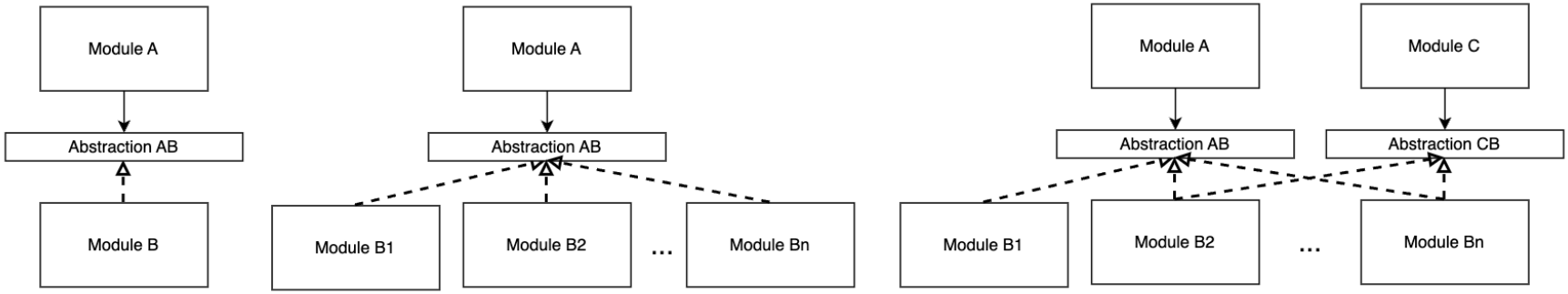
Eg: Module A (higher level) uses features of module B (lower level) in their operations, BUT module A should **not depend** on Module B. And both should depend on Abstraction AB.

DIP: The Dependency Inversion Principle

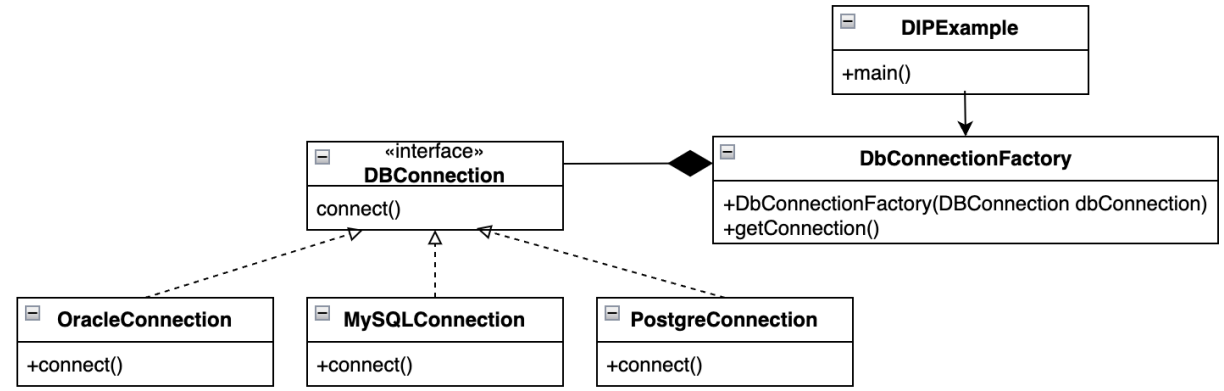


- Declarations:

> - *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
> - *Abstractions should not depend upon details. Details should depend upon abstractions.*



DIP: Example 1




```java
interface DBConnection {
    void connect();
}

class OracleConnection implements DBConnection {
    @Override
    public void connect() {
        System.out.println("Oracle connected");
    }
}

class MySQLConnection implements DBConnection {
    @Override
    public void connect() {
        System.out.println("MySQL connected");
    }
}
```

```java
class PostgreSQLConnection implements DBConnection {
    @Override
    public void connect() {
        System.out.println("PostgreSQL connected");
    }
}

class DbConnectionFactory {
    private final DBConnection dbConnection;

    public DbConnectionFactory(DBConnection dbConnection) {
        this.dbConnection = dbConnection;
        this.dbConnection.connect();
    }

    public DBConnection getConnection() {
        return this.dbConnection;
    }
}

public class DIPExample {

    public static void main(String[] args) {
        DBConnection conn = new OracleConnection();
        DbConnectionFactory factory = new DbConnectionFactory(conn);
    }
}
```

DIP: Stable Abstractions



- Every change to an abstract interface corresponds to a change to its concrete implementations. Conversely, changes to concrete implementations do not always, or even usually, require changes to the interfaces that they implement. Therefore interfaces are less volatile than implementations.
- Good software designers and architects work hard to **reduce the volatility of interfaces**. They try to find ways to add functionality to implementations without making changes to the interfaces.

DIP: Stable Abstractions



- Coding practices:
  - Don’t refer to volatile concrete classes.
  - Don’t derive from volatile concrete classes.
  - Don’t override concrete functions.
  - Never mention the name of anything concrete and volatile.

DIP: Exercise 1



- Students give an examples of stable interfaces and use **Adapter pattern** to connect between incompatible “stable” interfaces.

DIP: Factories

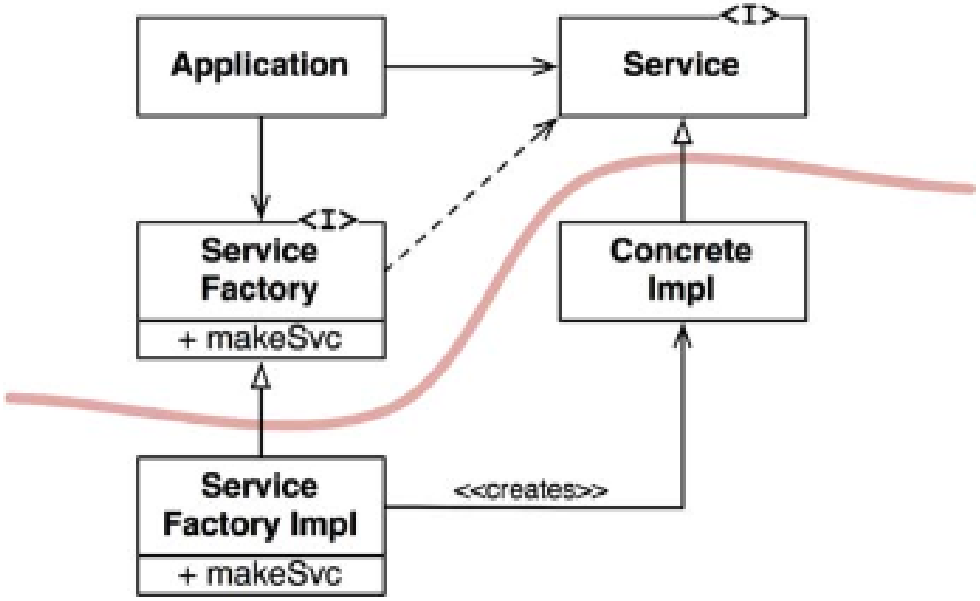


- To comply with these rules, the creation of volatile concrete objects requires special handling. This caution is warranted because, in virtually all languages, the creation of an object requires a source code dependency on the concrete definition of that object.
- In most object-oriented languages, such as Java, we would use an **Abstract Factory** to manage this undesirable dependency.

DIP: Factories



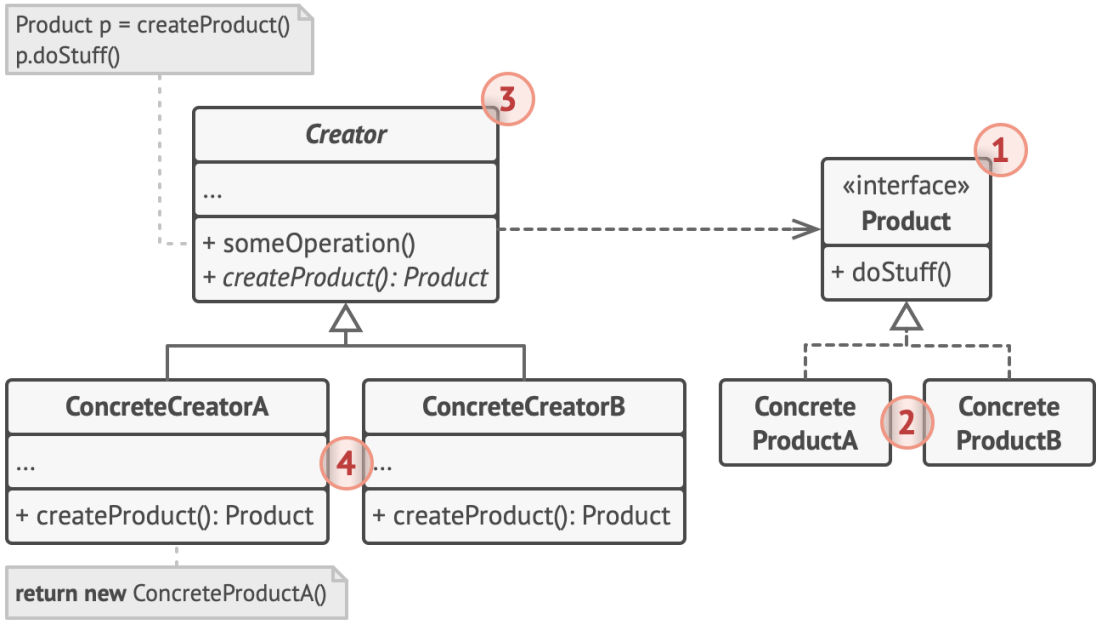
- Eg: Use of the Abstract Factory pattern to manage the dependency.
- The (red) curved line is an architectural boundary. It separates the abstract from the concrete. All source code dependencies cross that curved line pointing in the same direction, toward the abstract side.



DIP: Factory Method Pattern



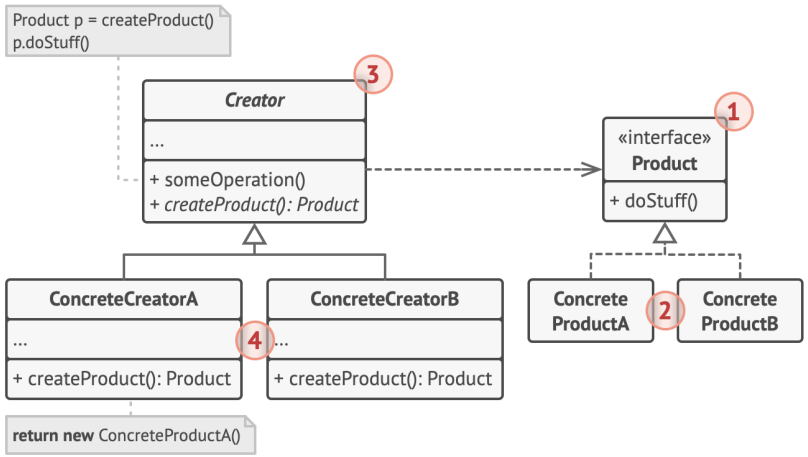
- Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



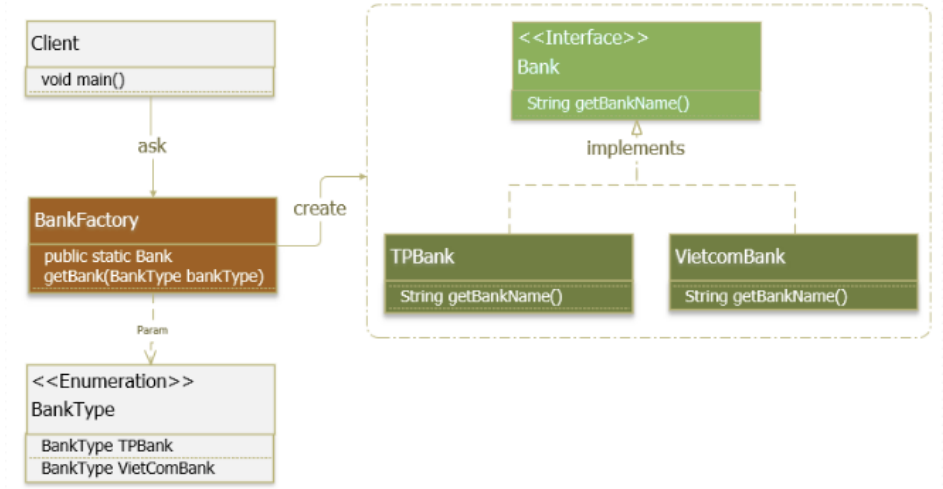
DIP: Factory Method Pattern



1. The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.
2. **Concrete Products** are different implementations of the product interface.
3. The **Creator** class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.
4. **Concrete Creators** override the base factory method so it returns a different type of product.



DIP: Example 2




```java
public enum BankType {
    VIETCOMBANK, TPBANK;
}

public class BankFactory {
    private BankFactory(){}

    public static final Bank getBank(BankType bankType) {
        switch (bankType) {
        case TPBANK:
            return new TPBank();
        case VIETCOMBANK:
            return new VietcomBank();
        default:
            throw new IllegalArgumentException("unsupported bankType");
        }
    }
}
```

```java
public class TPBank implements Bank {
    @Override
    public String getBankName() {
        return "TPBank";
    }
}

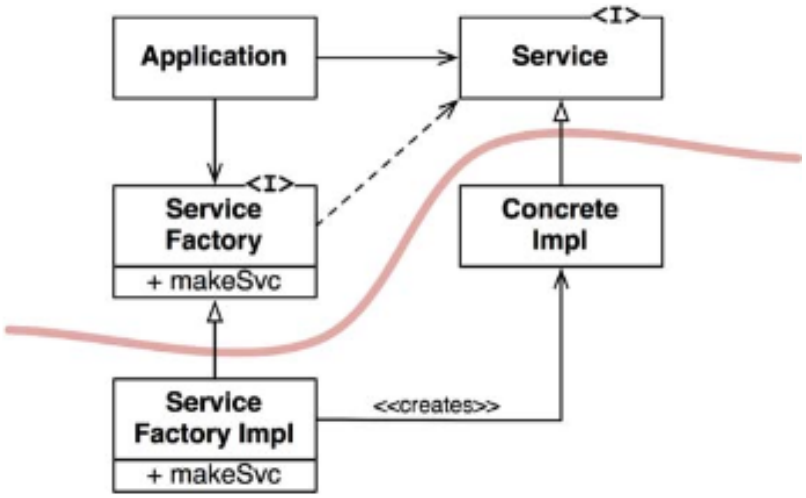
public class VietcomBank implements Bank {
    @Override
    public String getBankName() {
        return "VietcomBank";
    }
}
```

```java
public class Client {
    public static void main(String[] args) {
        Bank bank = BankFactory.getBank(BankType.TPBANK);
        System.out.println(bank.getBankName()); // TPBank
    }
}
```

DIP: Conclusion



- As we have more higher-level architectural components, DIP will be the most visible organizing principle in our architecture diagrams. The curved line (as figure) will become the architectural boundaries. The way the **dependencies cross that curved line in one direction**, and toward more abstract entities, will become a new rule that we will call the **Dependency Rule.**



DIP: Exercise 2



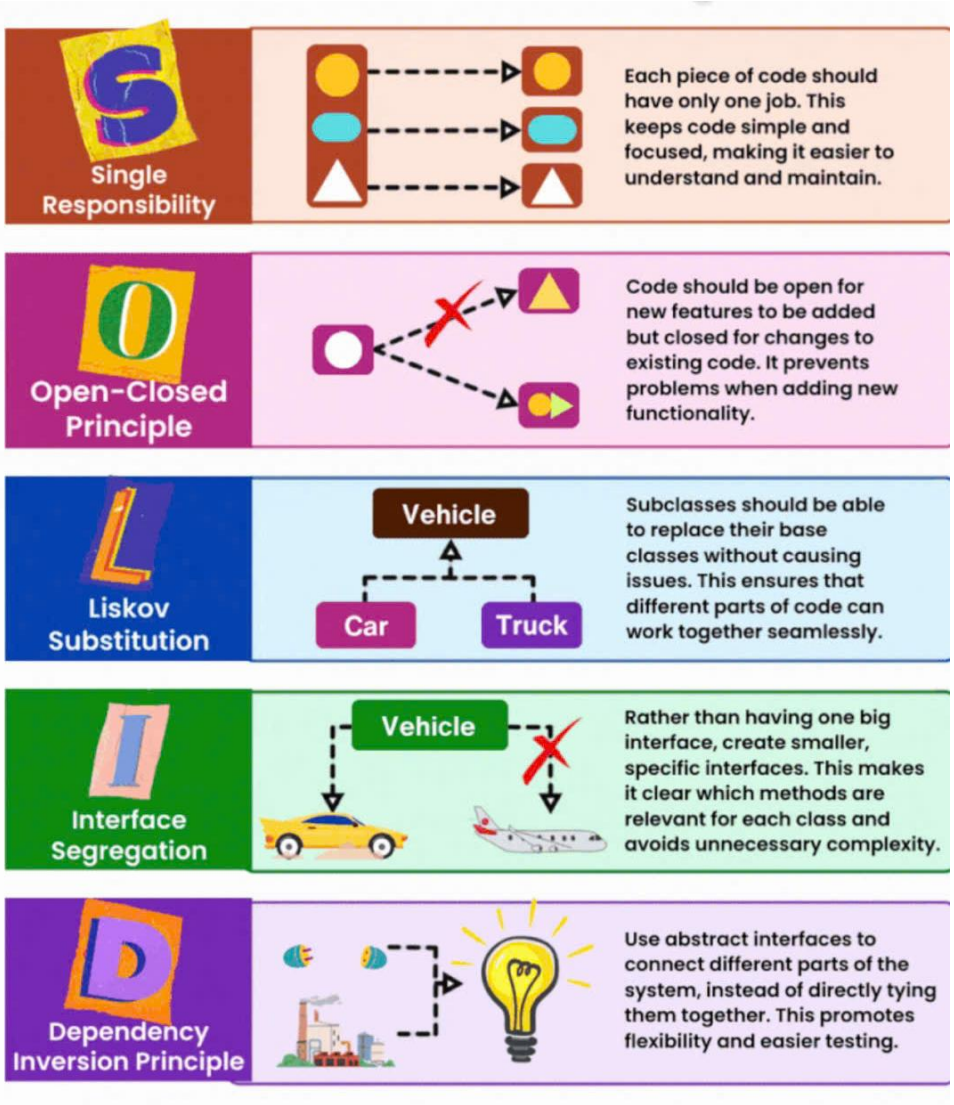
Students compare **Abstract Class** and **Interface** for implementing Factory design pattern.

DIP: Exercise 3



Students compare Factory Method, Abstract Factory Method and Builder design pattern.

Conclusion


S
Single Responsibility
Each piece of code should have only one job. This keeps code simple and focused, making it easier to understand and maintain.
O
Open-Closed Principle
Code should be open for new features to be added but closed for changes to existing code. It prevents problems when adding new functionality.
L
Liskov Substitution
Vehicle
Car
Truck
Subclasses should be able to replace their base classes without causing issues. This ensures that different parts of code can work together seamlessly.
I
Interface Segregation
Vehicle
Rather than having one big interface, create smaller, specific interfaces. This makes it clear which methods are relevant for each class and avoids unnecessary complexity.
D
Dependency Inversion Principle
Use abstract interfaces to connect different parts of the system, instead of directly tying them together. This promotes flexibility and easier testing.

Conclusion



Q&A